

Construye un gemelo digital del beamline

Un proyecto con SIMION y Optuna

Taller MIT-UNAL, Medellín

1. El objetivo

Cada grupo va a construir un **gemelo digital** de un pequeño beamline de iones simulado en SIMION. Un gemelo digital es un *emulador*: un modelo que recibe los voltajes de los electrodos y predice lo que hace el haz, lo bastante rápido y lo bastante exacto para usarlo en lugar de la simulación real cuando uno diseña y optimiza.

Por qué importa: una corrida fiel de SIMION es lenta, y la máquina real lo es mucho más. Un buen gemelo responde en milisegundos, se puede derivar, y permite optimizar sobre él sin pagar el costo de cada medición. El valor del gemelo no está en que reproduzca sus datos. Está en que les deje tomar decisiones correctas sobre la máquina sin tener que correrla. Un modelo que ajusta bien la tabla de entrenamiento pero manda a un mal punto de operación no sirve como gemelo.

Trabajan en **grupos de tres** y tienen **dos semanas**. Al final nos envían dos cosas: el **código** que produce su gemelo y resuelve las tareas de control, y un **informe en PDF** que explica el modelo que eligieron y la metodología que prefirieron. No se trata de un catálogo de todo lo que probaron, sino de un solo modelo bien hecho y bien justificado.

Para construir el gemelo primero tienen que conocer el beamline y saber correrlo en SIMION, porque es el mismo SIMION que van a usar para generar los datos de entrenamiento y para verificar su gemelo. Eso es lo que explican las secciones siguientes. La sección final describe el reto en detalle.

2. El beamline

El montaje es un beamline de iones: se lanza un haz por un extremo y el trabajo es dirigirlo y enfocarlo para que lleguen al detector tantos iones como sea posible. La medida clásica de qué tan bien lo hace el beamline es la **transmisión**:

$$\text{Transmisión} = \frac{\text{iones que llegan al detector}}{\text{iones lanzados (500)}} \times 100 \, \%.$$

La transmisión es solo una de las cantidades que pueden mirar. SIMION puede reportar mucho más que un conteo: la posición de llegada de cada ion, su velocidad, y su trayectoria completa. De ahí se derivan la transmisión, la distribución espacial del haz en el detector, su dispersión, o la velocidad de los iones. En el reto ustedes eligen qué cantidad emula su gemelo y para qué optimizan. Esa decisión se explica en la última sección.

Lo que cambia el comportamiento del haz son los *voltajes* de los electrodos, y hay varios. Probar combinaciones a mano no sirve, y por eso usarán una librería de optimización que busca buenos voltajes con SIMION como árbitro.

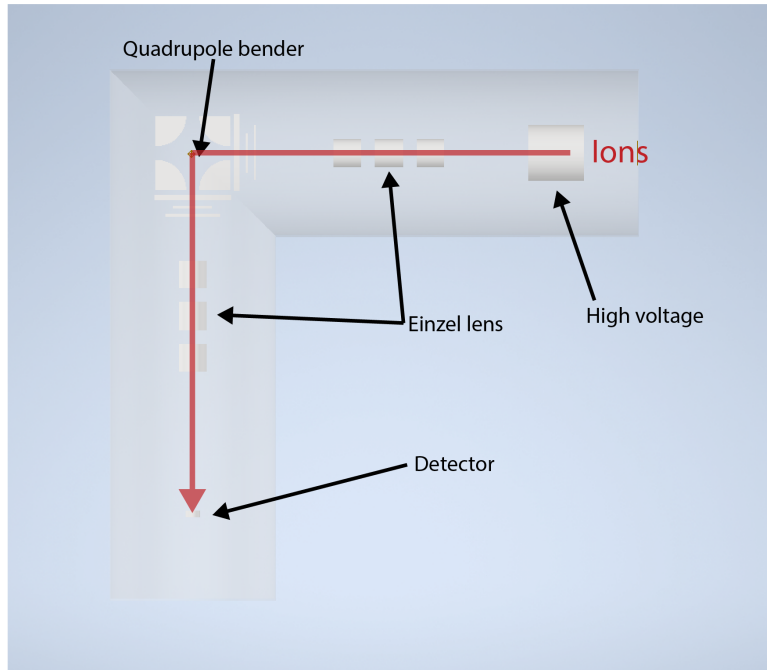


Figura 1: El beamline. Los iones nacen dentro de la jaula de alto voltaje, a la derecha, y viajan hacia la izquierda a través de dos lentes Einzel, que enfocan el haz. El curvador cuadrupolar gira el haz 90° en la esquina, y los iones bajan luego hasta el detector. En coordenadas de simulación el haz parte en (395, 75, 77) mm moviéndose a lo largo de $-x$, se curva hacia $+z$, y se recoge en una caja pequeña alrededor de (76, 76, 405) mm.

3. Los archivos de la simulación

Todos estos archivos viven juntos en una misma *carpeta de proyecto*, la misma donde guardan `optimizer.py`. El optimizador encuentra esa carpeta solo y usa rutas *relativas*, así que no importa dónde quede en su computador (la letra de unidad y la ruta serán distintas en cada máquina). Esto es para qué sirve cada tipo de archivo.

3.1. Archivos STL: la geometría

Los archivos `electrode_1.stl`, `electrode_2.stl`, ... son modelos 3D (exportados de CAD) de la *forma* de cada electrodo metálico: la jaula de alto voltaje, el tubo, los anillos de las lentes, el curvador, las placas y el detector. Describen *dónde está el metal*, pero no a qué voltaje está. SIMION lee estas formas para armar la simulación.

3.2. Los archivos .PA# y .PA0: los campos eléctricos

Un *arreglo de potencial* (PA) es la grilla con que SIMION describe el campo eléctrico en el espacio. Hay tres clases, y solo la primera se las entregamos:

- `electrode_.PA#`: el archivo de *definición*. Guarda la geometría (qué puntos de la grilla son metal, y a qué número de electrodo pertenece cada uno) pero **todavía no** el campo eléctrico. Es el único archivo del que parten. *Los otros dos los generan ustedes a partir de él* “refinando” (ver el Paso 0 en la Sección 5).

- `electrode_.PA1, electrode_.PA2, ..., electrode_.PA19`: un campo *resuelto* por electrodo, calculado con ese electrodo a voltaje unitario y todos los demás a tierra. Resolverlos es el paso lento, pero solo hay que hacerlo una vez.
- `electrode_.PA0`: el arreglo combinado de *ajuste rápido* (fast-adjust). Como los campos se suman de forma lineal, SIMION puede tomar el voltaje que ustedes eligen para cada electrodo y construir *al instante* el campo total escalando y sumando los arreglos por electrodo, sin volver a resolver nada. Eso es lo que permite probar miles de combinaciones de voltaje rápido. Los archivos `.PA1...PA19` y `.PA0` son grandes (cientos de MB cada uno), y por eso no se las enviamos; los crean una sola vez en su máquina refinando el pequeño `.PA#`.

3.3. El archivo `.fly2`: el haz de iones

`SimpleSetUp.fly2` define las partículas que disparamos: **500** iones de silicio con una sola carga ($^{28}\text{Si}^+$, masa 28 amu, carga +1), con una energía cinética de unos 15 eV, lanzados desde la posición (395, 75, 77) mm en un cono de 15° apuntando a lo largo de $-x$. Cada corrida vuela este mismo haz, así que cualquier cambio en la transmisión se debe a los *voltajes*, no al haz.

3.4. El archivo `.iob`: el banco de trabajo

`SimpleSetUp.iob` es el “banco de trabajo” (workbench) de SIMION: amarra todo (la geometría, los arreglos de potencial, el haz y la configuración de grabación) en una sola simulación ejecutable. Cuando le decimos a SIMION que “vuele”, este es el archivo al que apuntamos. Si quieren grabar algo distinto del conteo, por ejemplo la velocidad o las trayectorias, es aquí donde se ajusta la configuración de grabación.

4. Qué voltajes son fijos y cuáles se optimizan

No todos los electrodos son una perilla libre. Algunos *deben* quedarse en un voltaje fijo por el papel que cumplen (archivo `ElectrodesMaps.txt`):

- **Jaula de alto voltaje (+500 V)**. Es el recinto cargado donde se crean los iones. Su potencial fija la energía inicial del haz, así que es una propiedad fija de la fuente, no una perilla de ajuste.
- **Tubo de tierra y placas de tierra (0 V)**. Son los electrodos de referencia (“tierra”) y los blindajes electrostáticos. La tierra *es* el cero de nuestra escala de voltaje; cambiarla no dirigiría el haz, solo movería la referencia.
- **Detector (−2000 V)**. El detector se mantiene muy negativo para atraer y recoger los iones positivos. Es el instrumento de medida al final de la línea, así que su voltaje también es fijo.

Todo lo que *le da forma* al haz (las lentes, el curvador y las placas de dirección) queda libre. **Esos son los voltajes que se optimizan.** En `ElectrodesMaps.txt` son justo los electrodos que *no* tienen un valor escrito al lado. Son ocho, así que es una búsqueda de 8 dimensiones.

Entonces el optimizador es libre de elegir voltajes para los electrodos **3, 6, 9, 10, 11, 12, 15 y 18**; todos los demás quedan fijos en los valores de arriba en cada corrida.

Mantengan los voltajes físicos. Para cada electrodo libre tienen que darle al optimizador un *rango* de voltajes donde buscar. Elijan esos rangos para que coincidan con lo que de verdad puede entregar una fuente de laboratorio; no pueden sacar millones de voltios. Como regla práctica, ± 500 V está disponible con comodidad, y ± 1000 V **es el máximo que deberían permitir**. Un voltaje que la fuente no puede entregar es inútil, aunque a la simulación “le guste”, así que no dejen que la búsqueda se salga de ahí. (La fuente fija de +500 V y el detector de −2000 V corren con sus

Tabla 1: Los electrodos del beamline.

Electrodo(s)	Componente	Voltaje
1	Jaula de alto voltaje (fuente de iones)	+500 V (fijo)
2	Tubo de tierra	0 V (fijo)
3	Lente Einzel 1 (elemento polarizado)	optimizar
4, 5	Lente Einzel 1 (elementos a tierra)	0 V (fijo)
6	Lente Einzel 2 (elemento polarizado)	optimizar
7, 8	Lente Einzel 2 (elementos a tierra)	0 V (fijo)
9 a 12	Curvador cuadrupolar (cuatro partes)	optimizar
13, 14	Placas de tierra	0 V (fijo)
15	Placa de voltaje / dirección 1	optimizar
16, 17	Placas de tierra	0 V (fijo)
18	Placa de voltaje / dirección 2	optimizar
19	Detector	−2000 V (fijo)

propias fuentes de alto voltaje dedicadas, y por eso a esos dos se les permite estar fuera del rango de ± 1000 V.)

5. Correr SIMION desde la terminal

Una vez SIMION está configurado, casi todo pasa por unas pocas instrucciones de línea de comandos. El **Paso 0 lo corren una sola vez** al comienzo; el optimizador corre después **los Pasos 1 y 2 de forma automática** en cada intento que hace (y también pueden correr esos dos a mano para ver qué hacen).

Asegúrense primero de que “simion” se pueda ejecutar. Todo comando de abajo empieza con la palabra `simion`. Para que su terminal la entienda, el computador tiene que saber dónde está instalado SIMION, y esa carpeta es distinta en cada máquina. Si escriben `simion` y ven un error como “simion is not recognized” (Windows) o “command not found”, elijan una de estas:

- **Agreguen SIMION al PATH.** Agreguen la carpeta de instalación de SIMION (la que contiene `simion.exe`, normalmente `C:\Program Files\SIMION-8.1`) a la variable de entorno *Path* del sistema, y abran una *nueva* terminal. Ahora `simion` a secas funciona en todas partes, tal cual aparece abajo.
- **O escriban la ruta completa al programa** en cada comando, en lugar de la palabra `simion`. En PowerShell ponen el operador de llamada `&` delante de la ruta entre comillas, por ejemplo:

```
& "C:\Program Files\SIMION-8.1\simion.exe" --nogui refine "electrode_.PA#"
```

(El optimizador de Python ya sabe dónde está SIMION por el ajuste `SIMION_INSTALL_DIR` de su `CONFIG`, así que esto solo importa para los comandos que escriban *ustedes* a mano.)

El manual de SIMION. La referencia más fácil es el PDF que viene con esta actividad, `simion8manual.pdf`. Documenta una versión más vieja de SIMION, pero las herramientas de línea de comandos son las mismas, y están todas descritas en el **Apéndice M (Command Line Interface)**; busquen ahí cada opción de `refine`, `fastadj` y `fly`.

Su SIMION instalado también trae el manual más nuevo y completo como `simion.chm` en la carpeta de SIMION (la que tiene `simion.exe`); búsquen ahí “command line”. Tengan en cuenta que `.chm` es un archivo de ayuda de Windows, y Windows a menudo lo **bloquea** (por Smart App Control, o por la “Mark of the Web” del archivo), así que puede abrir en blanco o decir “Navigation to the webpage was canceled”. Para permitirlo: clic derecho en `simion.chm` → *Properties* → marquen *Unblock* → *OK*. Si Smart App Control lo sigue bloqueando, cópielo a una carpeta local como el Escritorio y ábralo desde ahí.

Paso 0: refinar el arreglo de potencial (refine), una sola vez. Solo les damos el archivo de geometría `electrode_.PA#`. “Refinar” es el paso donde SIMION resuelve el campo eléctrico de la geometría y escribe los archivos pesados de campo: `electrode_.PA0` más los arreglos por electrodo `electrode_.PA1...PA19`. Córranlo una vez desde la carpeta del proyecto:

```
simion --nogui refine "electrode_.PA#"
```

Esto puede tardar unos minutos y algo de memoria; es normal, porque están resolviendo un campo en una grilla de millones de puntos. Solo se hace una vez; después de eso los archivos de campo existen y volar es rápido. Si cambian la *geometría*, refinan de nuevo; si solo cambian *voltajes*, no.

Paso 1: fijar los voltajes (fastadj). El subcomando `fastadj` escribe un voltaje en cada electrodo del arreglo de ajuste rápido `.PA0`. Recibe el archivo del arreglo seguido de una lista de pares `electrodo=voltaje` separados por comas. *Armar este comando es su Tarea 1b en `optimizer.py`.*

Paso 2: volar los iones y ver dónde caen (fly). El subcomando `fly` manda los 500 iones por el campo definido en el banco de trabajo e imprime dónde cae cada uno. *Armar este comando es su Tarea 1a en `optimizer.py`.*

Ambos comandos los arman ustedes, con las piezas de la tabla de abajo y el manual de SIMION (`simion8manual.pdf`, Apéndice M; ver la nota de arriba). Esto es lo que hace cada pieza:

Después del vuelo, SIMION imprime una línea de la forma `xyz(76, 75, 405)mm` por cada ion que llega al detector. El optimizador cuenta esas líneas; ese conteo es la transmisión. Si graban más datos, esas mismas líneas traen la posición y, si lo configuran, la velocidad de cada ion.

6. El optimizador de partida

Una vez hecho el Paso 0 (refinado `electrode_.PA#`, así que los archivos de campo existen), corren un solo programa de Python, `optimizer.py`:

```
python optimizer.py
```

Por dentro repite este ciclo (un método llamado *optimización bayesiana*):

1. **Optuna propone** un voltaje para cada uno de los 8 electrodos libres (dentro de rangos que pueden editar al inicio del archivo).
2. El script **fija** esos voltajes con `fastadj`.
3. SIMION **vuela** los 500 iones con `fly`.
4. El script **cuenta** cuántos iones llegaron al detector; ese conteo es el puntaje.
5. Optuna **aprende** del puntaje y propone un conjunto de voltajes más inteligente la próxima vez.

Tabla 2: Qué significa cada parte del comando de SIMION.

Parte del comando	Qué hace
<code>simion</code>	Lanza el programa SIMION.
<code>--nogui</code>	Lo corre <i>sin interfaz gráfica</i> (headless) para que un script lo maneje automáticamente.
<code>fly</code>	El subcomando que de verdad simula (“vuela”) los iones por el campo.
<code>fastadj</code>	El subcomando que fija al instante los voltajes de los electrodos, usando el arreglo de ajuste rápido <code>.PA0</code> .
<code>.^electrode_.PA0"</code>	El archivo de arreglo de potencial cuyos voltajes estamos fijando. (Entre comillas porque la ruta de la carpeta tiene un espacio.)
<code>1=500,2=0,...</code>	La lista de asignaciones <code>electrodo=voltaje</code> . Fijamos <i>todos</i> los electrodos cada vez para que cada corrida sea reproducible.
<code>--recording-output=out.txt</code>	El archivo donde SIMION escribe los datos que graba.
<code>--retain-trajectories=0</code>	<i>No</i> guardar la trayectoria completa de cada ion en memoria, ya que solo necesitamos dónde termina. Más rápido y más liviano. (Si su gemelo modela trayectorias, pónganlo en 1.)
<code>--restore-potential=0</code>	<i>No</i> deshacer los voltajes después de la corrida; dejar puesto el campo que acabamos de fijar.
<code>--programs=0</code>	Correr el vuelo por lotes sin pasadas extra de programas de usuario, para que cada corrida sea ligera.
<code>"SimpleSetUp.iob"</code>	El banco de trabajo (geometría + haz + grabación) que estamos volando.

Al final de todas las pruebas, el mejor conjunto de voltajes es el resultado de la optimización. Los resultados también se guardan en `beamline_results.csv`, una tabla de voltajes y lo que produjeron. Esa tabla es justo la semilla del conjunto de entrenamiento del gemelo: la búsqueda y la emulación son dos mitades del mismo proceso, porque el lugar donde decidieron medir determina lo que el modelo puede aprender.

Para qué les sirve este optimizador en el reto. Es dos cosas a la vez. Es un *generador de datos*: cada corrida les da un par de voltajes y su resultado, y dónde decide medir la búsqueda moldea su conjunto de entrenamiento. Y es una *forma de optimizar*, una de las que pueden usar después para la tarea de control sobre su gemelo.

Cosas para experimentar:

- Cambien N_TRIALS: ¿una búsqueda más larga encuentra una mejor respuesta?
- Amplíen o estrechen los rangos de voltaje en OPTIMIZE (pero manténganlos dentro del límite físico de ± 1000 V).
- Cambien OBJECTIVE de "hits" a "spread" para apuntar al haz *más estrecho* en el detector en vez de a la mayor cantidad de iones. Es un primer ejemplo de que pueden optimizar para algo distinto del conteo.
- Denle al optimizador un buen STARTING_POINT para arrancar en caliente.

7. El reto: el gemelo digital

Aquí está el proyecto. Lo que es libre es de ustedes; lo que es fijo es lo que el gemelo tiene que cumplir y cómo lo evaluamos. El puntaje premia un gemelo útil, no uno que memorice puntos.

7.1. Qué construyen

Un emulador que reciba un vector de voltajes y devuelva la cantidad que ustedes eligen modelar. La arquitectura es decisión de ustedes. Algunas opciones razonables:

- un proceso gaussiano, como el que vieron en la parte de optimización, ahora en más dimensiones,
- una red neuronal pequeña,
- una regresión sobre variables construidas a mano, por ejemplo proyecciones o productos de voltajes,
- un ensamble de varios modelos,
- un modelo que incorpore lo que saben de la física del montaje.

Exploren con libertad. Pueden probar varias arquitecturas, varias estrategias de datos y varios métodos de optimización durante las dos semanas. Pero entregan **un solo modelo**: el que prefirieron, con la metodología que los llevó a él. El informe no es un catálogo de todo lo que intentaron, sino la defensa del modelo que eligieron.

7.2. Qué cantidad modelan

SIMION reporta más que el conteo. De cada corrida pueden sacar la transmisión (el número de iones en el detector), la distribución espacial de las posiciones de llegada, la dispersión o el tamaño del haz, la velocidad de los iones, o sus trayectorias. **Ustedes eligen qué cantidad emula su gemelo y para qué optimizan.** Pueden quedarse con la transmisión, apuntar a un haz bien enfocado, o modelar una salida con varias componentes. Si eligen una cantidad que el montaje no graba por defecto, ajusten la configuración de grabación del banco de trabajo (por ejemplo, retener trayectorias o grabar velocidad). Justifiquen su elección en el informe.

7.3. Recolección de datos

Tienen un SIMION que pueden correr, pero cada corrida cuesta, así que hay un **presupuesto de evaluaciones** para entrenar. El instructor lo fija al comienzo. Decidir dónde medir es una decisión

de diseño: pueden usar la optimización bayesiana del optimizador, un muestreo que llene el espacio, una grilla, un muestreo aleatorio, o aprendizaje activo guiado por la incertidumbre de su propio modelo. La idea es sacarle al modelo la mayor información posible por cada evaluación gastada. Las corridas que hagan para verificar su gemelo no cuentan contra el presupuesto de entrenamiento.

7.4. Las dos tareas de control

Estas dos tareas miden si el gemelo sirve para controlar la máquina, que es el punto de todo el ejercicio.

- **Dirección.** Usen su gemelo para encontrar los voltajes que llevan su cantidad al mejor valor (por ejemplo, máxima transmisión, o mínimo tamaño del haz). Reporten esos voltajes, córranlos una vez en SIMION, y reporten el resultado verdadero. Medimos qué fracción del óptimo real alcanzaron.
- **Consigna inversa.** Les damos un valor objetivo de su cantidad. Usen su gemelo para hallar los voltajes que producen ese objetivo, córranlos en SIMION, y reporten el resultado. Medimos qué tan cerca quedó el resultado verdadero del objetivo.

La primera tarea premia un gemelo certero cerca del óptimo. La segunda premia un gemelo certero en un rango, no solo en un punto.

7.5. Cómo evaluamos el gemelo

Un solo número, entre cero y uno. Cada término está normalizado para no depender de la escala de la cantidad que eligieron, sino de qué tan bueno es su gemelo.

$\text{puntaje} = G \cdot (0.30 C + 0.20 A + 0.15 I + 0.15 E + 0.10 D + 0.10 F)$
--

- **C, dirección.** El valor verdadero en el punto que recomienda su gemelo, como fracción del óptimo de referencia que el instructor conoce. Es el término de mayor peso porque es la razón de ser del gemelo.
- **A, exactitud en la región informativa.** Qué tan bien predicen, medido solo donde la cantidad es informativa, por ejemplo donde de verdad llegan iones al detector. Lo medimos ahí a propósito, porque en un problema con señal escasa un modelo que predice cero en todas partes parece bueno y no lo es.
- **I, consigna inversa.** Qué tan cerca del objetivo quedó el resultado verdadero de la tarea de consigna, normalizado contra una propuesta sin modelo.
- **E, extrapolación y honestidad.** Qué tanto se degrada el modelo fuera de la región donde entrenó, y si su incertidumbre crece donde debe crecer. Premia fallar con elegancia y avisar que no sabe. Penaliza estar seguro y equivocado.
- **D, diferenciabilidad.** Si se puede obtener el gradiente del gemelo y si ese gradiente concuerda con una diferencia finita de SIMION cerca de un buen punto de operación. Un modelo del que no se puede sacar gradiente saca cero aquí.
- **F, eficiencia de datos.** Premia alcanzar una exactitud mínima usando menos evaluaciones de SIMION que el presupuesto.
- **G, compuerta de admisibilidad.** Un multiplicador menor que uno si el gemelo predice valores no físicos, por ejemplo conteos negativos o fuera de rango. Así se exige que el gemelo respete la física, no como consejo sino como condición.

Conviene que el grupo estime estos términos por su cuenta antes de entregar. Todos se pueden calcular con sus propias verificaciones y con los resultados de control que de todos modos van a reportar.

7.6. Reglas

- Separen datos para verificar el gemelo y no los usen para entrenar. Si los mismos pares entran al entrenamiento, la exactitud deja de medir nada.
- Respeten el presupuesto de evaluaciones para entrenar. Anoten cuántas usaron.
- El gemelo debe ser determinista al predecir y devolver valores admisibles, es decir físicos y dentro de rango.
- Fijen las semillas de sus modelos y de su recolección de datos, para que sus resultados se puedan reproducir desde el código.

7.7. Qué entregan

Dos cosas, por grupo de tres.

El código. Que sea ejecutable y reproduzca su gemelo y sus resultados de control desde los datos que recolectaron. Con las semillas fijas, alguien debería poder correrlo y obtener lo mismo.

El informe en PDF, sobre el modelo que eligieron, con figuras. Una guía de lo que esperamos:

1. La cantidad que eligieron modelar y por qué.
2. Recolección de datos: qué método usaron, cuántas evaluaciones, y una figura de cómo quedó cubierto el espacio.
3. El modelo: arquitectura, hiperparámetros, y cómo lo validaron.
4. Resultados: predicho contra real en la región informativa, el comportamiento fuera de la región de entrenamiento, y la incertidumbre si la tienen.
5. Control: el resultado de la tarea de dirección y de la consigna inversa, con figuras, y el método de optimización que usaron sobre el gemelo.
6. La metodología que prefirieron: por qué eligieron este camino y no otro.
7. Reflexión: hasta dónde confían en su gemelo y dónde no.

7.8. Consejos y errores comunes

- Hagan funcionar todo el flujo de punta a punta con un modelo sencillo antes de refinar. Un gemelo mediocre pero completo, temprano, vale más que un modelo elegante a medias al final.
- Escalen las entradas y las salidas antes de ajustar el modelo. Voltajes y conteos viven en rangos muy distintos, y casi todos los métodos, en particular los procesos gaussianos y las redes, fallan si no se escalan.
- No gasten todo el presupuesto en una sola tanda. Midan algo, ajusten un primer modelo, miren dónde falla, y recojan los siguientes puntos ahí.
- Cuidado con la región escasa. Si recolectan solo al azar, casi todos sus puntos van a tener señal cero y el modelo no aprenderá dónde está la señal.
- Revisen que su gemelo nunca prediga valores no físicos. Si puede pasar, recórtelo, o modelen el logaritmo de la cantidad cuando tenga sentido.